NigeriaCompliance Workflow - Template-Based Dynamic Styling

Version: 2.1
Date: May 2026
Architecture: FastAPI Backend + Vercel Frontend + Railway Deployment

---

Table of Contents

Executive Summary

The NigeriaCompliance system is a cloud-native compliance reporting platform that processes organizational documents and generates beautifully formatted reports with **dynamic styling based on user-provided templates**.

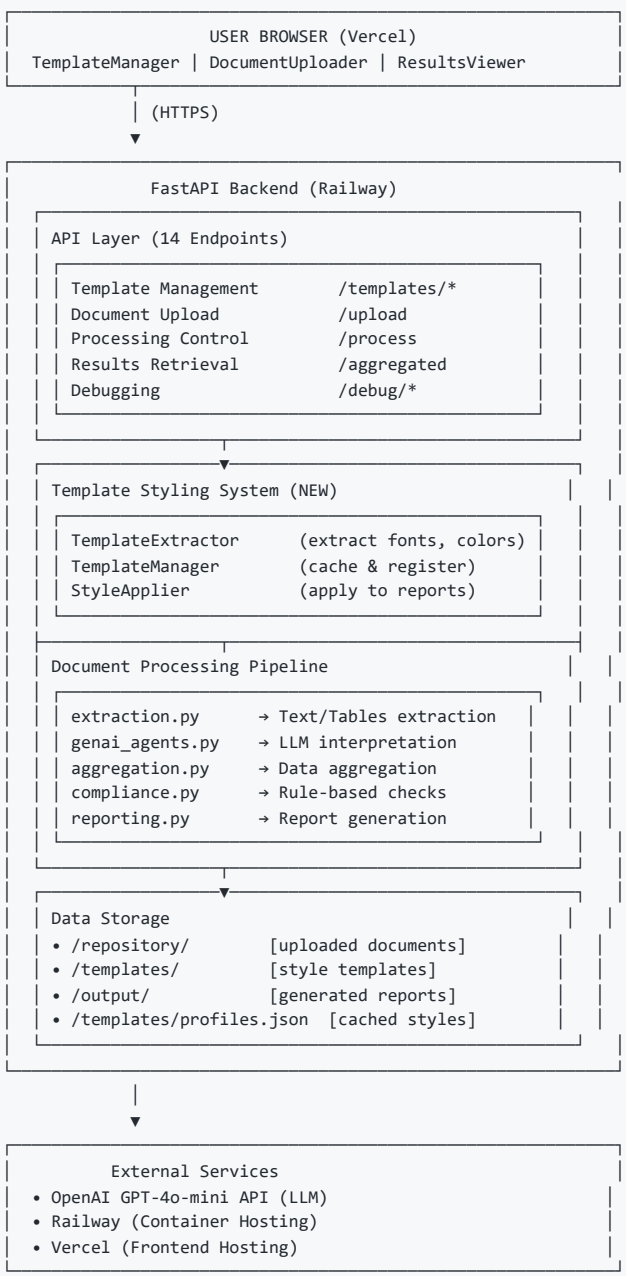
Key Features

✅ **Template-Based Styling**: Upload a style template, and all generated reports match that styling
✅ **Multi-Format Support**: Process Word, PDF, Excel, and CSV documents
✅ **AI-Powered Analysis**: Uses OpenAI GPT-4o-mini for intelligent data extraction
✅ **Cloud-Native**: Deployed on Railway with auto-scaling
✅ **Incremental Processing**: Only processes new documents since last run
✅ **RESTful API**: 14+ endpoints for full workflow automation

---

System Architecture

Component Overview



New Template-Based Workflow

Workflow Steps

Step 1: Upload Style Template (NEW)

User uploads a Word document or PDF that serves as the style template.

```
INPUT:  professional_template.docx (contains custom fonts, colors, margins)
        ↓
PROCESS: TemplateExtractor extracts styling:
  • Title font:   Calibri, 28pt, bold, color #003366
  • Heading font: Calibri, 16pt, bold
  • Body font:    Calibri, 12pt
  • Margins:      1" all sides
  • Page layout:  Letter size, portrait
        ↓
OUTPUT: Template profile cached in /templates/profiles.json
        API response: Extracted styling information
```

API Call:

```
POST /templates/upload
Content-Type: multipart/form-data
file: professional_template.docx
template_name: "professional"

Response:
{
  "success": true,
  "template_name": "professional",
  "styles": {
    "title_font": {"name": "Calibri", "size": 28, "bold": true},
    "heading_font": {"name": "Calibri", "size": 16, "bold": true},
    "body_font": {"name": "Calibri", "size": 12}
  }
}
```

Step 2: Activate Template (NEW)

User selects which template to use for processing.

```
INPUT:  Template name: "professional"
        ↓
PROCESS: TemplateManager validates template exists
         Sets active_template in process status
        ↓
OUTPUT: Template activated and ready for use
```

API Call:

```
POST /templates/professional/activate

Response:
{
  "success": true,
  "active_template": "professional",
  "message": "Template 'professional' activated"
}
```

Step 3: Upload Data Documents (Existing)

User uploads documents to be processed.

```
INPUT:  finance_report.xlsx, hr_document.docx, etc.
        Department: Finance, HR, Procurement, Operations
        ↓
PROCESS: Files stored in /repository/{department}/
         Tracked in .processed_files.json
        ↓
OUTPUT: Files ready for processing
```

API Call:

```
POST /upload
Content-Type: multipart/form-data
file: finance_report.xlsx
department: "finance"

Response:
{
  "stored_path": "/repository/finance/uuid_finance_report.xlsx",
  "department": "finance",
  "filename": "uuid_finance_report.xlsx"
}
```

Step 4: Process with Template Styling (MODIFIED)

System processes documents and applies template styling to outputs.

```
INPUT:  Active template: "professional"
        Unprocessed files in /repository/
        ↓
PROCESSING PIPELINE:

  1. EXTRACTION (extraction.py)
     Extract text, tables from documents
     Support: DOCX, PDF, XLSX, CSV, TXT

  2. INTERPRETATION (genai_agents.py → interpretation_agent)
     INPUT:  Raw extracted text/tables
     PROMPT: "Identify department, period, and key metrics"
     OUTPUT: {department, period, metrics, confidence}

  3. AGGREGATION (aggregation.py)
     Combine metrics from multiple documents
     Calculate totals and summaries

  4. COMPLIANCE CHECKS (compliance.py)
     Rule-based validation
     Identify issues and risks

  5. RISK ANALYSIS (genai_agents.py → risk_analysis_agent)
     INPUT:  Aggregated data + compliance issues
     PROMPT: "Analyze risks and provide recommendations"
     OUTPUT: Risk narrative and analysis

  6. EXECUTIVE SUMMARY (genai_agents.py → report_writer_agent)
     INPUT:  Aggregated data + risk analysis
     PROMPT: "Write executive summary with findings"
     OUTPUT: Formatted narrative for report

  7. REPORT GENERATION WITH TEMPLATE STYLING (**NEW**)
     INPUT:  Template profile + aggregated data
     PROCESS: StyleApplier applies template to PDF
     • Uses template fonts for titles, headings, body
     • Applies template margins and spacing
     • Applies template color scheme
     OUTPUT: Professional PDF matching template
        ↓
OUTPUT: HTML and PDF reports in /output/
        Reports styled exactly like template document
```

API Call:

```
POST /process

Response:
{
  "success": true,
  "message": "Processing started in the background",
  "running": true,
  "status_url": "/process/status",
  "aggregated_url": "/aggregated"
}
```

Step 5: Retrieve Results (Existing)

User downloads generated reports.

```
OUTPUT FILES:
  • aggregated_data.json        [structured data]
  • Financial_Compliance_Report_Q1_2025.html
  • Financial_Compliance_Report_Q1_2025.pdf  [WITH TEMPLATE STYLING]
  • Summary_Revenue_vs_Payroll.png
  • Summary_Vendor_Spend.png
```

API Endpoints

Template Management (NEW)

Endpoint	Method	Purpose
`/templates/upload`	POST	Upload and register style template
`/templates`	GET	List all registered templates
`/templates/{name}/activate`	POST	Activate template for processing
`/templates/info/{name}`	GET	Get template styling details

Core Workflow

Endpoint	Method	Purpose
`/upload`	POST	Upload document for processing
`/process`	POST	Start background processing
`/process/status`	GET	Check processing status
`/aggregated`	GET	Get aggregated results
`/artifact/{filename}`	GET	Download generated file
`/health`	GET	Health check

Debugging

Endpoint	Method	Purpose
`/debug/files`	GET	List all uploaded files
`/debug/test-extraction`	GET	Test extraction on sample file
`/debug/env`	GET	View environment variables
`/debug/output`	GET	List output files
`/debug/processing-files`	GET	Show files to be processed

Workflow Modules

1. Template Styling (`workflow/template_styling.py`) NEW

Purpose: Extract, manage, and apply template styling

Classes:

- `FontStyle` : Represents font properties (name, size, bold, italic, color)
- `ParagraphStyle` : Paragraph properties (alignment, spacing, indent)
- `SectionStyle` : Section-level styling
- `TemplateProfile` : Complete styling profile from template
- `TemplateExtractor` : Extracts styling from DOCX/PDF
- `StyleApplier` : Applies styling to generated documents
- `TemplateManager` : Manages template registration and caching

Key Methods:

```
# Extract styling from template
extractor = TemplateExtractor()
profile = extractor.extract("template.docx")

# Register template
manager = TemplateManager()
manager.register_template("template.docx", "professional")

# Apply styling to document
applier = StyleApplier(profile)
styled_doc = applier.apply_to_docx(doc)
```

2. Document Extraction (`workflow/extraction.py`)

Purpose: Extract text and tables from multiple document formats

Supported Formats:

- DOCX (Word)
- PDF (with OCR fallback)
- XLSX (Excel)
- CSV
- TXT
- Images (with Tesseract OCR)

Returns:

```
{
  "raw_text": str,           # Extracted text
  "raw_tables": List[Dict],  # Extracted tables
  "source_path": str,        # File path
  "file_type": str           # docx, pdf, xlsx, etc.
}
```

3. LLM Agents (`workflow/genai_agents.py`)

Purpose: AI-powered data interpretation

Interpretation Agent

Prompt: "Extract department, period, and metrics from this document"

Output:

```
{
  "department": "Finance",
  "period": "Q1 2025",
  "metrics": {
    "revenue": 1000000,
    "expenses": 750000,
    "profit_margin": 0.25
  },
  "confidence": 0.95,
  "missing_fields": [],
  "notes": []
}
```

Risk Analysis Agent

Prompt: "Analyze compliance risks in this data"

Output:

```
Key risks identified:
1. Revenue variance exceeds threshold
2. Expense-to-revenue ratio suboptimal
3. Missing documentation for procurement
```

Report Writer Agent

Prompt: "Write executive summary from this data"

Output:

```
NigeriaCompliance Financial Review - Q1 2025

The organization's financial position remains stable with
revenue of $1M and profit margin of 25%. Key risks include
variance in year-over-year revenue...
```

LLM Configuration:

```
# Priority 1: OpenAI API (if OPENAI_API_KEY set)
# Priority 2: Local Ollama (fallback)

# Environment Variables:
OPENAI_API_KEY = "sk-..."
OPENAI_MODEL = "gpt-4o-mini"  # Default
LLM_MAX_RETRIES = 3
LLM_CALL_TIMEOUT = 60
LLM_BACKOFF_FACTOR = 1.5
```

4. Aggregation (`workflow/aggregation.py`)

Purpose: Combine data from multiple documents

Input: List of records from interpretation agent

Output:

```
{
  "metrics": {
    "total_revenue": 5000000,
    "total_expenses": 3750000,
    "total_payroll": 1000000,
    "total_vendor_spend": 500000
  },
  "departmental": {
    "Finance": {"metrics": {...}},
    "HR": {"metrics": {...}},
    "Operations": {"metrics": {...}},
    "Procurement": {"metrics": {...}}
  },
  "notes": [],
  "data_quality": 0.92
}
```

5. Compliance (`workflow/compliance.py`)

Purpose: Rule-based compliance checking

Rules:

1. Revenue must be > 0
2. Expense-to-revenue ratio < 0.9
3. All departments must be represented
4. No negative metrics allowed
5. Payroll must be reasonable % of revenue

Output:

```
(
  status="COMPLIANT",  # or "NON_COMPLIANT", "WARNINGS"
  issues=[
    "Q1 revenue below Q4 baseline",
    "HR department missing data"
  ]
)
```

6. Reporting (`workflow/reporting.py`) MODIFIED

Purpose: Generate reports with optional template styling

Functions:

```
def create_summary_charts(aggregated, output_dir) -> Dict[str, Path]:
    # Generate PNG charts for visualizations
    # Returns: {chart_name: chart_path}

def generate_html_report(...) -> Path:
    # Generate HTML report (unchanged)
    # Returns: path_to_html

def generate_pdf_report(..., template_profile=None) -> Path:
    # NEW: Generate PDF with template styling applied
    # Uses template fonts, colors, margins if provided
    # Returns: path_to_pdf
```

Template Styling Application (in PDF generation):

```
# Before: Static fonts
c.setFont("Helvetica", 10)

# After: Template-based fonts
if template_profile:
    title_font = template_profile.title_font.name
    title_size = template_profile.title_font.size
    title_color = template_profile.title_font.color
    c.setFont(title_font, title_size)
```

7. Main Workflow (`workflow/run_workflow.py`) MODIFIED

Purpose: Orchestrate entire processing pipeline

Function Signature (NEW parameter):

```
def process_repository(
    base_dir: Optional[str] = None,
    repo_dir: Optional[str] = None,
    output_dir: Optional[str] = None,
    template_name: Optional[str] = None  # NEW
) -> Dict:
    """
    Process repository with optional template styling
    """
```

Returns:

```
{
    "aggregated": {...},
    "status": "COMPLIANT",
    "issues": [...],
    "risk_narrative": "...",
    "narrative": "...",
    "charts": {...},
    "aggregated_json": "path",
    "html_report": "path",
    "pdf_report": "path",
    "template_used": "professional"  # NEW
}
```

LLM Integration

Model Selection

Current Configuration: OpenAI GPT-4o-mini

Rationale:

- Fast response times (< 5 seconds)
- Strong performance on document analysis
- Cost-effective ($0.15 per 1M input tokens)
- Reliable for structured JSON output

Alternatives Considered:

- GPT-4: Too slow for batch processing (~60+ seconds)
- GPT-3.5: Inconsistent data extraction
- Local Ollama: Good for simple tasks, struggles with complex analysis

LLM Prompts

Interpretation Agent Prompt

```
You are a financial compliance expert. Extract the following from the provided document:
1. Department (Finance, HR, Procurement, Operations)
2. Financial period (e.g., "Q1 2025")
3. Key metrics (revenue, expenses, headcount, vendor spend)
4. Missing fields that should be present
5. Confidence score (0.0-1.0)

Respond ONLY with valid JSON.
```

Risk Analysis Agent Prompt

```
You are a compliance auditor. Analyze the provided financial data for risks:
1. Identify threshold violations
2. Flag unusual trends
3. Highlight missing documentation
4. Recommend corrective actions

Provide a concise narrative assessment.
```

Report Writer Agent Prompt

```
You are an executive communications expert. Write a 200-word executive summary:
1. Current compliance status
2. Key findings
3. Risk assessment
4. Recommendations

Use professional business language.
```

Deployment

Railway Deployment Configuration

Dockerfile:

```
FROM python:3.11-slim

# Install system dependencies
RUN apt-get update && apt-get install -y \
    tesseract-ocr \
    poppler-utils \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy and install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY . .

# Expose port (Railway assigns dynamically)
EXPOSE ${PORT:-8000}

# Run application
CMD ["uvicorn", "api.server:app", "--host", "0.0.0.0", "--port", "${PORT:-8000}"]
```

Environment Variables (Railway):

```
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4o-mini
FRONTEND_ORIGINS=https://your-vercel-app.vercel.app
OLLAMA_BASE_URL=http://localhost:11434
PORT=8000
```

Auto-Deployment:

- Push to GitHub → Railway auto-rebuilds
- New environment deployed automatically
- No manual deployment required

Vercel Frontend Deployment

Configuration (`vercel.json`):

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "env": [
    "REACT_APP_API_URL"
  ]
}
```

Environment (Vercel):

```
REACT_APP_API_URL=https://your-railway-app.up.railway.app
```

Testing

Test Suite

Unit Tests (`test_template_styling.py`):

- ✅ Template extraction from DOCX
- ✅ Template registration and caching
- ✅ Style application to documents
- ✅ Workflow integration

Integration Tests:

```
# 1. Extract template
python -m pytest tests/test_extraction.py

# 2. Process documents
python -m pytest tests/test_workflow.py

# 3. API endpoints
python -m pytest tests/test_api.py

# 4. Full end-to-end
python -m pytest tests/test_e2e.py
```

Manual Testing Workflow

```
# 1. Start server
uvicorn api.server:app --reload

# 2. Upload template
curl -X POST http://localhost:8000/templates/upload \
  -F "file=@templates/professional_template.docx" \
  -F "template_name=professional"

# 3. Activate template
curl -X POST http://localhost:8000/templates/professional/activate

# 4. Upload data
curl -X POST http://localhost:8000/upload \
  -F "file=@sample_data.xlsx" \
  -F "department=finance"

# 5. Process
curl -X POST http://localhost:8000/process

# 6. Check status
curl http://localhost:8000/process/status

# 7. Get results
curl http://localhost:8000/aggregated > results.json

# 8. Download PDF (with template styling)
curl http://localhost:8000/artifact/Financial_Compliance_Report_Q1_2025.pdf \
  -o report.pdf
```

Troubleshooting

Common Issues

Issue: "No templates found"

```
Solution:
1. Verify template upload endpoint is working
2. Check /templates/profiles.json exists
3. Confirm file format is .docx or .pdf
```

Issue: "Processing fails with template"

```
Solution:
1. Check template profile is valid
2. Verify template_name matches active template
3. Review Railway logs for styling errors
4. Try processing without template first
```

Issue: "OpenAI API key not recognized"

```
Solution:
1. Check OPENAI_API_KEY env var is set
2. Verify key format: sk-...
3. Use /debug/env endpoint to confirm
4. Restart Railway deployment after setting vars
```

Issue: "Extracted text is empty or incorrect"

```
Solution:
1. Check file format is supported
2. For PDFs, verify OCR is working
3. Use /debug/test-extraction endpoint
4. Check document quality and resolution
```

Performance Metrics

Operation	Duration	Notes
Template extraction	0.5-2s	Depends on document size
Document processing	10-30s	Includes LLM calls
Report generation	2-5s	With chart generation
PDF styling	1-2s	With template applied
Total workflow	**15-40s**	Per document set

Future Enhancements

1. **Template Previews**: Visual preview of template styling before activation
2. **Template Customization UI**: Edit styling through web interface
3. **Multi-Language Support**: Generate reports in multiple languages
4. **Data Visualization**: Enhanced chart and graph generation
5. **Batch Processing**: Upload multiple templates and documents at once
6. **Template Marketplace**: Share templates between organizations
7. **Audit Trail**: Track template changes and document processing history
8. **Webhooks**: Real-time notifications for processing events

---

Support & Contact

For issues or questions:

1. Check this documentation
2. Review `/debug/*` endpoints
3. Check Railway deployment logs
4. Review test suite for examples

Documentation Version History:

- v1.0 (Jan 2026): Initial workflow documentation
- v2.0 (Mar 2026): FastAPI backend, Railway deployment
- v2.1 (May 2026): Template-based dynamic styling system